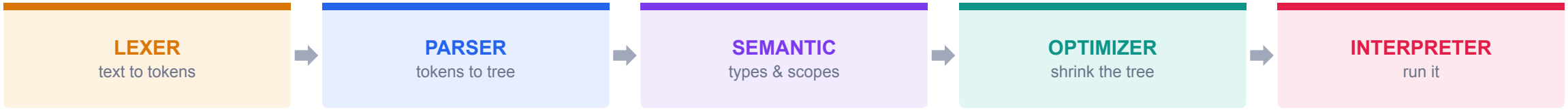
COMPILER DESIGN · PROJECT REVIEW

Lucid

A compiler you can **see through**.

Built from scratch in C++17 · live playground · AI assistant



5	3	2	0
PHASES	TYPES	AI ACTIONS	FRAMEWORKS

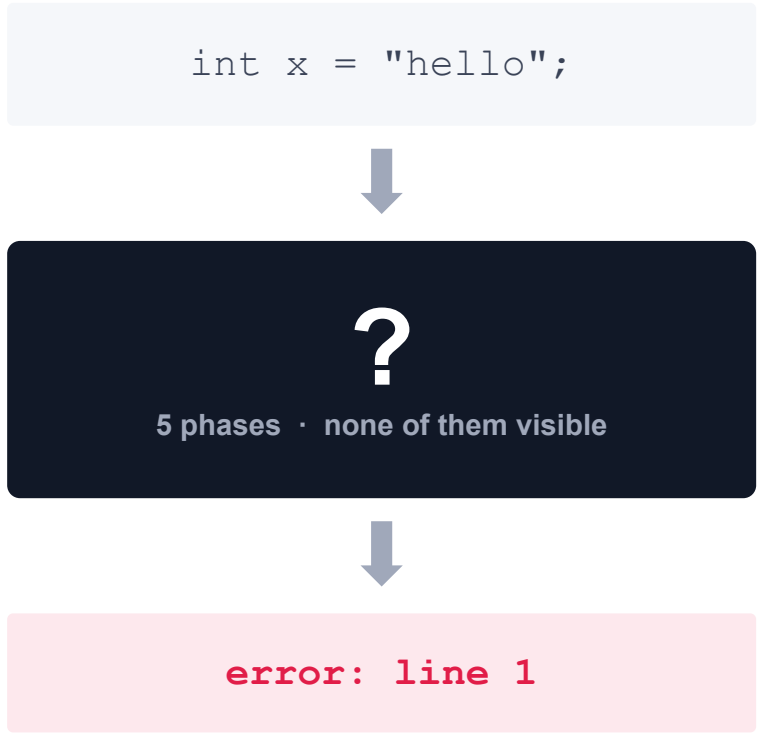
THE PROBLEM

A compiler is a black box

- You never see the tokens.
- You never see the syntax tree.
- You never see the symbol table.

And when it fails, you get a line number.
Never which phase rejected your code, or why.

> **Internals are invisible.**
> **Errors are unexplained.**

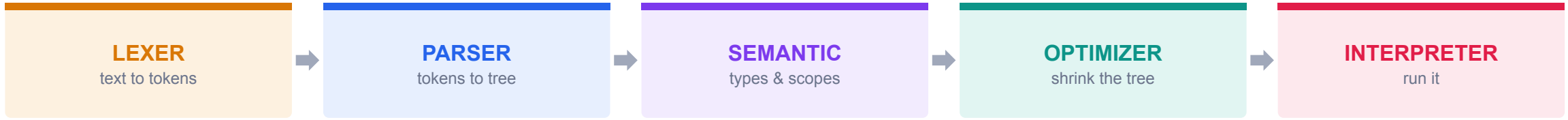


THE SOLUTION

Lucid opens the box

A real compiler that shows its own internals — live, as you type.

FIVE PHASES



EVERY ONE OF THEM VISIBLE



Type a character — every panel refreshes in 300 milliseconds.

OBJECTIVES

What we set out to build

Build it by hand

No Lex. No Yacc. No frameworks.

Show every stage

Each phase renders its own output.

Name the phase that failed

Lexical, syntax, semantic or runtime.

Optimise and prove it

A measured before and after count.

Report all errors at once

Never stop at the first mistake.

AI explains, never decides

The compiler has the final say.

SCOPE

Where we drew the line

WE BUILT

- A statically-typed language
- All five phases, hand-written
- Type checking and a symbol table
- Constant folding and dead-code removal
- Four kinds of error, clearly named
- A playground and a command-line tool

WE DID NOT

- Generate machine code
- Emit three-address code
- Support floats, arrays or classes
- Add modules or imports
- Deploy it to the web

Four phases done thoroughly beats six phases done shallowly.

ARCHITECTURE

Three layers, one direction

PLAYGROUND
Browser · five live panels · AI assistant

one request · one JSON reply

SERVER
A thin C++ shell — no compiler logic

THE COMPILER



Pure C++17

No network. No framework.

Runs two ways

In a browser, or in a terminal.

AI sits outside

Remove it — nothing breaks.

HOW IT WORKS

Source in, five stages, output out

1

LEXER

Reads the text

Splits it into tokens.

2

PARSER

Builds the tree

Checks the grammar.

3

SEMANTIC

Checks meaning

Types, names and scopes.

4

OPTIMIZER

Shrinks the tree

Removes dead code.

5

INTERPRETER

Runs the program

Prints the output.

Break your code and the pipeline stops at the phase that rejected it — you see exactly where.

COMPILER CONCEPTS

The theory, running

Tokens and maximal munch	Lexer
Grammar and recursive descent	Parser
Operator precedence	Parser
Panic-mode error recovery	Parser
Symbol table and scopes	Semantic
Static type checking	Semantic
Constant folding	Optimizer
Dead-code elimination	Optimizer
Activation records	Interpreter

Implemented — and visible while it runs.

FEATURES

What makes it different

AI

AI explains errors

Plain English, on demand.

AI

AI fixes your code

Then the compiler re-checks it.

< >

Live playground

Five panels, updating as you type.

AST

Visual syntax tree

Watch the tree take shape.

55%

Real optimisation

38 nodes become 17.

?

Did you mean...?

Spelling help on unknown names.

THE AI ASSISTANT

It explains. It never decides.

1

SEND

Your broken code
and our error

2

SUGGEST

The AI proposes
a correction

3

VERIFY

Our compiler
re-checks it

Repeats up to three times — until it compiles.

The AI never decides whether your code is correct.
That verdict always comes from the compiler.

Optional — the whole project works without it.

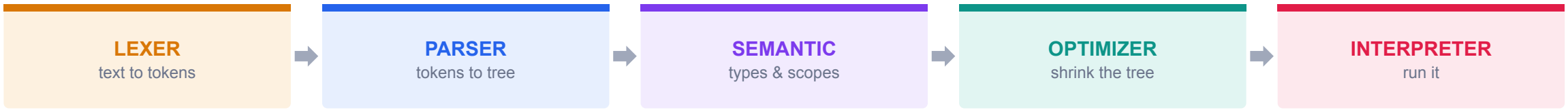
DEMO

What you will see

1	**Write a program**	Tokens and the tree appear instantly.
2	**Delete a semicolon**	It stops at the syntax stage.
3	**Break the types**	Three errors, all reported at once.
4	**Misspell a name**	It asks: did you mean 'count'?
5	**Run the optimizer**	38 nodes become 17, side by side.
6	**Press Fix it**	The AI repairs it, the compiler confirms.

THANK YOU

Lucid makes a compiler visible while it runs.



Questions